

PROJECT REPORT

EduGenie

Google Gemini Powered Learning Assistant

Field	Details
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Team ID	SWTID-2026-1665
Date	02 October 2026

1. INTRODUCTION

1.1 Project Overview

EduGenie is an AI-powered learning assistant built on Google Gemini, Google's multimodal generative AI model. It gives students a single place to ask questions, get topic explanations at their own level, generate summaries and study notes, and practise with quizzes and flashcards created from their own study material.

Instead of returning a list of links like a search engine, EduGenie holds a conversation. It breaks complex concepts into simple steps, gives examples, adapts to follow-up questions, and responds in a clear, encouraging tone. The assistant is delivered as a web application so that it can be used from any device with a browser.

Main capabilities:

- **Concept explanation:** step-by-step answers adjusted to the learner's level (beginner, intermediate, advanced).
- **Summaries and notes:** condenses long text or chapters into concise, revision-ready notes.
- **Quiz Arena:** creates quizzes on any topic with instant grading and a performance history.
- **Doubt solving:** supports follow-up questions in a continuous conversation.
- **Learning paths, notes and analytics:** personalised roadmaps, saved notes and bookmarks, and a dashboard of quiz performance.

1.2 Purpose

The purpose of this project is to make quality learning support available to every student, at any time, without the cost or scheduling limits of private tutoring. EduGenie aims to:

- Provide instant, accurate and easy-to-understand explanations of academic topics.
- Encourage active learning through quizzes and practice instead of passive reading.
- Personalise the explanation style and difficulty to each learner.
- Reduce the time students spend searching across many sources for a single answer.
- Demonstrate a practical, responsible use of generative AI in education.

1.3 Application Scenarios and Use Cases

Scenario	User	How EduGenie helps
Exam preparation	School and college students	Explains difficult topics, creates quizzes and flashcards, and builds a revision plan before exams.
Homework and doubt solving	Students at any level	Gives step-by-step explanations at any hour, with follow-up questions in the same conversation.
Self-paced learning	Self-learners, job seekers	Breaks a new subject into a study path and checks understanding with practice questions.

Lesson material support	Teachers and tutors	Drafts simple explanations, examples and practice questions that teachers can review and reuse.
Revision from own notes	Any learner	Condenses pasted notes into short revision points and key terms.
Learning in simple language	Non-native English speakers	Rephrases complex content in simple, clear language.

1.4 Project Objectives

- Build a working web application that uses the Google Gemini API to act as a personal tutor.
- Design effective system instructions and prompts so that answers are accurate, structured and age-appropriate.
- Support the core learning modes: asking questions, concept explanation, text summarisation, quizzes with instant grading and learning paths.
- Let learners choose a level (beginner, intermediate, advanced) so the explanation matches their background.
- Keep the API key secure and handle errors so that the application never crashes in front of the user.
- Test the system for functionality, output quality, speed and user acceptance, and document the results.

1.5 Prerequisites and Prior Knowledge

Prerequisite	Details
Software	Python 3.10 or higher, pip, a code editor (VS Code), Git
Libraries	fastapi, uvicorn, Google Gemini SDK, python-dotenv (confirm against your requirements.txt)
Accounts	Google account with a Gemini API key from Google AI Studio; GitHub account
Hardware	Any laptop or desktop with 4 GB RAM and an internet connection
Browser	Latest Chrome, Edge or Firefox

Prior knowledge required:

- Basic Python programming (functions, dictionaries, string formatting, exception handling).
- Basic understanding of APIs and JSON.
- Awareness of generative AI concepts such as large language models, prompts, tokens and temperature.
- Basic use of Git and GitHub.

1.6 System Workflow

Step	Description
1	The learner opens the web app, selects a level and a learning mode, and types a topic, question or pastes notes.
2	The application checks the input (not empty, within length limit) and builds a prompt from the chosen mode template plus recent conversation history.
3	The system instruction (EduGenie's tutor role and rules) and the prompt are sent to the Gemini API using the secured API key.
4	Gemini generates the response, which is checked for errors and formatted as Markdown.
5	The answer is shown in the chat window and saved in session history so the learner can ask follow-up questions.

1.7 Technical Architecture Overview

EduGenie is a three-layer application. The presentation layer (web interface) collects input and shows output. The application layer (Python FastAPI backend) validates input, manages session history and builds prompts. The AI layer (Google Gemini API) generates the educational content. The detailed architecture is described in Section 4.3 and the model choices in Section 4.4.

2. IDEATION PHASE

2.1 Problem Statement

Students often struggle to find clear, personalised help when they get stuck. Teachers cannot give one-to-one attention to every learner, private tutoring is expensive, and online search returns scattered, inconsistent information. The table below defines the customer problem statements using the standard template.

No.	I am	I'm trying to	But	Because	Which makes me feel
PS-1	A school or college student preparing for exams	Understand difficult concepts and revise efficiently	Explanations in books and videos are too generic or too advanced	I cannot get personalised help the moment I am stuck	Frustrated and anxious
PS-2	A self-learner studying a new subject online	Build a structured understanding and test my knowledge	I cannot tell what to study next or whether I have understood	Resources are scattered and give no feedback	Confused and unmotivated

2.2 Empathy Map Canvas

The empathy map captures what a typical EduGenie user (a student preparing for exams) says, thinks, does and feels.

Quadrant	Observations
Says	"I don't understand this topic." "Can someone explain it in a simpler way?" "I need more practice questions."
Thinks	"Am I falling behind my classmates?" "There is too much to cover before the exam." "I wish I had a tutor available right now."
Does	Searches many websites and videos, copies notes, asks friends, re-reads the same chapter, studies late at night.
Feels	Overwhelmed, anxious about results, frustrated when explanations are unclear, motivated when progress is visible.
Pains	No instant help, generic explanations, wasted time searching, no feedback on understanding, cost of tutoring.
Gains	Quick clear answers, personalised pace, confidence from practice, better marks, saved time, anytime access.

2.3 Brainstorming

The team brainstormed ideas for a learning solution and then prioritised them by impact and feasibility.

Idea	Description	Impact	Feasibility	Priority
Conversational tutor	Chat-based Q&A with explanations and follow-ups	High	High	Must have
Summariser	Turns long text into short notes	High	High	Must have
Quiz and flashcard generator	Creates practice questions from a topic or notes	High	High	Must have
Level-based explanations	Beginner / intermediate / advanced modes	Medium	High	Should have
Study planner	Builds a day-wise schedule for an exam	Medium	Medium	Should have
Multilingual support	Answers in the learner's preferred language	Medium	Medium	Could have
Progress dashboard	Tracks quiz scores and weak topics	Medium	Low	Future scope

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	Touchpoint	Experience	Opportunity
-------	-------------	------------	------------	-------------

Awareness	Student hears about EduGenie from peers or the institution	Social media, college portal	Curious	Clear value message
Onboarding	Opens the web app and selects subject and level	Landing page	Easy, quick	No sign-up barrier
Learning	Asks a question or pastes study material	Chat interface	Gets a clear answer	Simple language, examples
Practice	Requests a quiz or flashcards	Practice module	Engaged, challenged	Instant feedback
Review	Revisits summaries and weak topics	Notes and history	Confident	Personalised revision
Retention	Returns for the next topic or exam	Web app	Trusts the tool	Study planner, reminders

3.2 Solution Requirement

Functional requirements

ID	Requirement	Description
FR-1	Ask a question	User enters a question or topic in natural language and receives an explanation.
FR-2	Level selection	User chooses the explanation level (beginner, intermediate, advanced).
FR-3	Summarise content	User pastes or uploads text and receives a concise summary or notes.
FR-4	Quiz Arena	System generates quiz questions on a topic, grades answers instantly and stores the score history.
FR-5	Follow-up conversation	System keeps the context of the conversation for follow-up questions.
FR-6	Learning paths	System generates a personalised learning roadmap for a given topic.

Non-functional requirements

ID	Requirement	Description
NFR-1	Usability	Simple, clean interface usable without training on desktop and mobile browsers.
NFR-2	Performance	Responses are returned within a few seconds for typical queries.
NFR-3	Reliability	Graceful error messages if the API is unavailable or rate limited.
NFR-4	Security	API keys are kept in environment variables, never in the source code or client.

NFR-5	Privacy	No personal data is collected beyond what is needed for the session.
NFR-6	Safety	Content filters and prompt instructions keep answers educational and appropriate.
NFR-7	Scalability	Stateless backend design allows more users without redesign.

3.3 Data Flow Diagram

Level 0 (context): the Student sends a query or study material to EduGenie and receives explanations, summaries, quizzes or study plans in return. EduGenie exchanges prompts and responses with the Google Gemini API, an external entity.

Level 1 (main processes):

Step	Process	Input	Output
1	Capture user input	Question, topic, level or study text from the UI	Validated request
2	Build prompt	Validated request, conversation history, system instructions	Structured prompt
3	Call Gemini API	Structured prompt	Generated response
4	Post-process response	Generated response	Formatted answer, quiz items or notes
5	Display and store session	Formatted output	Rendered answer; updated chat history (session store)

Data stores: D1 quiz history and D2 notes and bookmarks. External entity: Google Gemini API.

Figure: DFD Level 0 (Context Diagram)

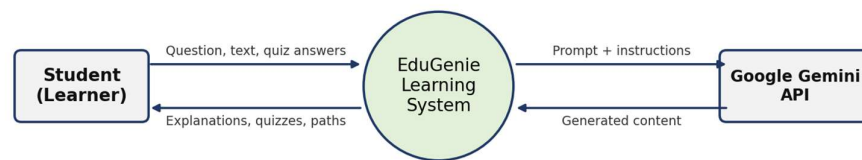


Figure 3.1: Data Flow Diagram, Level 0 (context diagram)

Figure: DFD Level 1

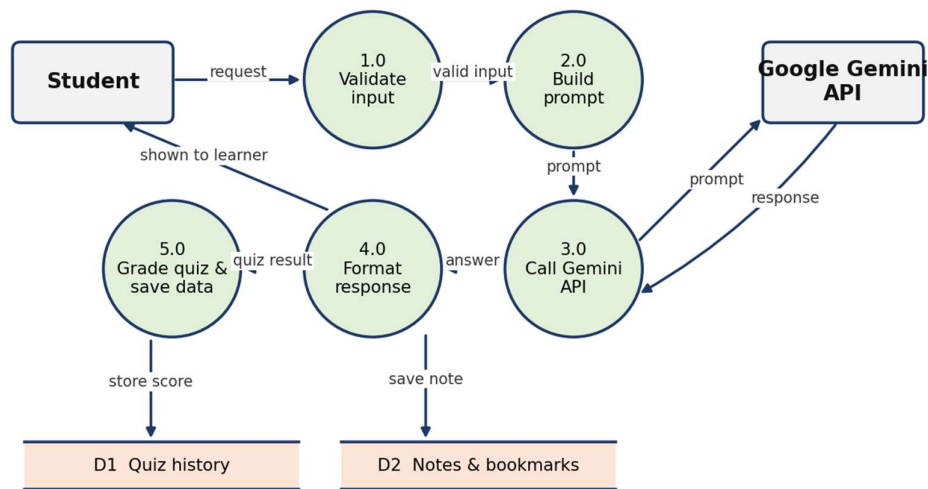


Figure 3.2: Data Flow Diagram, Level 1

3.4 Technology Stack

Layer	Technology	Purpose
AI model	Google Gemini 2.5 (via Gemini API / Google AI Studio)	Understanding queries and generating explanations, summaries and quizzes
Backend	Python with FastAPI	REST API, application logic, prompt construction, Gemini API calls, auto-generated API documentation
Frontend	HTML, CSS and JavaScript web interface	Learning Hub, Quiz Arena, Learning Paths, Notes & History, Analytics and API Docs pages
SDK	Google Gemini Python SDK	Communication with the Gemini API
Configuration	Environment variables / .env	Secure API key storage
Version control	Git and GitHub	Source code management
Deployment	Bolt hosting (bolt.host)	Public hosting of the web app

3.5 Data Collection and Preparation

EduGenie relies on the pretrained knowledge of Gemini rather than a custom-trained model, so the data work focuses on preparing user inputs and evaluation samples. The table below follows the project development guidelines.

Aspect	Approach in EduGenie
--------	----------------------

Data source identification	User inputs (questions, topics, pasted notes); a small test set of sample questions from school and college syllabi; optional text or CSV files of notes supplied by the learner.
Data collection method	Inputs are captured live through the chat interface; the test set was prepared manually by the team from open textbooks and class notes.
Data cleaning	Remove empty inputs, extra whitespace, duplicate lines and obvious noise (stray symbols, repeated characters) before sending text to the model.
Preprocessing and formatting	Trim long text to the allowed length, keep the mode template and user text separate, and format the prompt as clear instruction plus content.
Validation and quality check	Reject empty or over-length input; review a sample of test inputs to confirm that they are clean, relevant and correctly labelled by subject and level.
Privacy and ethical considerations	No personal data (name, phone, e-mail) is requested or stored; session history is kept only in memory; learners are told that AI answers should be verified; content filters are kept on.
Dataset documentation	Test set stored in the repository with columns: ID, subject, level, mode, input text, expected key points.

Sample input-output pairs used for prompting and testing:

Mode	Sample input	Expected output
Explain a concept	What is photosynthesis? (Beginner)	Simple definition, the equation in words, a plant-and-sunlight example, and a quick check question.
Summarise notes	Pasted notes on Newton's laws	Three short bullet points, one for each law, with key terms.
Create a quiz	Quiz on the Python list data type	Five MCQs with four options each and an answer key.
Make flashcards	Flashcards on the French Revolution	Eight question-answer cards covering causes, events and results.
Study plan	7-day plan for DBMS exam	Day-wise topics with revision and practice slots.

3.6 User Stories

ID	User type	User story	Acceptance criteria	Priority
US-1	Student	As a student, I want to ask a question in plain language so that I understand a topic quickly.	A clear explanation appears within a few seconds.	High
US-2	Student	As a student, I want to choose my level so that the explanation is neither too easy nor too hard.	Wording and depth change with the selected level.	High

US-3	Student	As a student, I want to paste my notes and get a summary so that I can revise faster.	Summary keeps all key points and is shorter than the input.	High
US-4	Student	As a student, I want a quiz on a topic so that I can test myself.	Quiz has questions, options and correct answers.	High
US-5	Self-learner	As a self-learner, I want a study plan so that I know what to study each day.	Plan lists topics per day for the chosen duration.	Medium
US-6	Teacher	As a teacher, I want to generate practice questions so that I can save preparation time.	Questions are relevant and editable by copying the text.	Medium
US-7	Any user	As a user, I want a friendly message when something fails so that I am not confused.	No crash; an error message and retry option are shown.	High

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem	How EduGenie solves it
Generic, one-size-fits-all explanations	Adapts explanation level and style to the learner.
No help available at the moment of doubt	Available 24/7 through the chat interface.
Time lost searching multiple sources	Gives a consolidated answer in one place.
No way to check understanding	Generates quizzes and flashcards with instant feedback.
High cost of private tutoring	Free or low-cost access to a personal learning assistant.

4.2 Proposed Solution

EduGenie is a web-based learning assistant that uses Google Gemini as its reasoning engine. The application wraps the model with education-focused prompt design: a system prompt defines EduGenie as a patient tutor that explains step by step, uses examples, checks understanding, and avoids unsafe or irrelevant content.

Aspect	Details
Novelty	Combines explanation, summarisation, practice and planning in a single conversational tool.
Personalisation	Learner level and subject selection shape every response.
Social impact	Widens access to quality tutoring for students who cannot afford it.
Business model	Free tier for students; institutional licences or premium features such as progress analytics.

Scalability	Stateless backend and a managed model API let usage grow without major rework.
-------------	--

4.3 Solution Architecture

EduGenie follows a simple three-layer architecture:

- **Presentation layer:** web interface where the student types questions, selects level and mode, and reads responses.
- **Application layer:** Python backend that validates input, applies the system prompt, manages conversation history and formats output.
- **AI layer:** Google Gemini API that generates the explanations, summaries, quizzes and study plans.

Flow: Student → Web UI → Application logic and prompt builder → Gemini API → Response formatter → Web UI → Student.

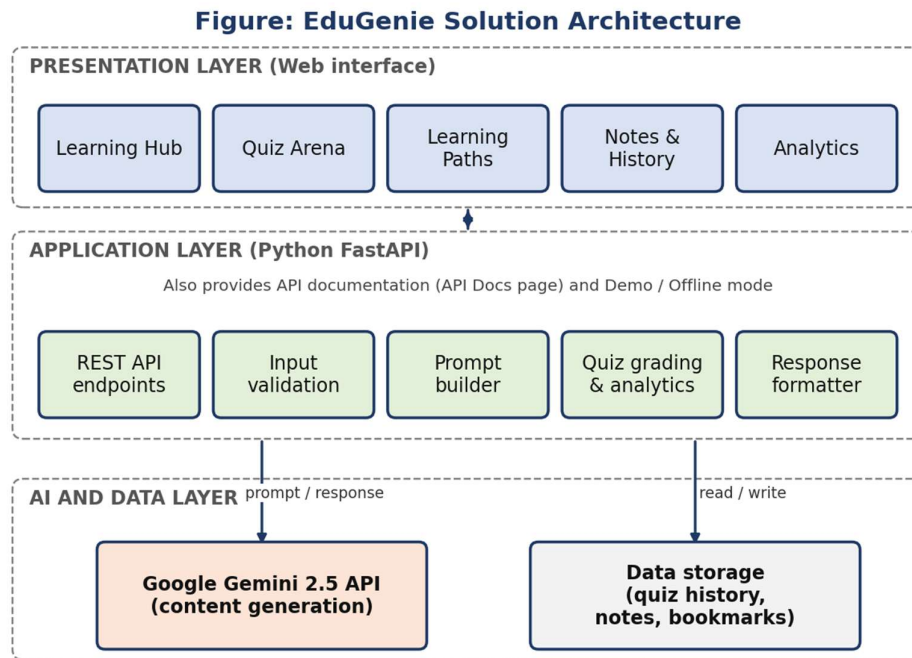


Figure 4.1: EduGenie solution architecture

4.4 Model Selection and Architecture Design

Option	Strengths	Limitations	Decision
Gemini Pro	Strongest reasoning, best for complex multi-step problems	Slower and costlier per request	Optional upgrade for advanced mode
Gemini Flash	Fast, low cost, good quality for explanations, summaries and quizzes	Slightly less depth on very hard reasoning tasks	Selected as default model

Vertex AI (Gemini)	Enterprise features, security controls, scaling on Google Cloud	More setup, billing and cloud configuration	Planned for production deployment
--------------------	---	---	-----------------------------------

Reasons for selecting Gemini Flash: learners expect quick replies, tutoring tasks (explain, summarise, quiz) do not need the heaviest model, the free tier in Google AI Studio suits a student project, and the API is simple to integrate from Python.

Design details:

- **Workflow:** User → Web UI → prompt builder (mode template + level + history) → Gemini API → response formatter → UI.
- **API integration:** The google-genai Python SDK is used; the API key is read from an environment variable and never exposed in the browser.
- **Token and context handling:** Only the last six messages are sent as context and long pasted notes are trimmed, which keeps requests within the model's context window and reduces cost and latency.
- **Libraries and tools:** FastAPI, Gemini SDK, python-dotenv, Git, GitHub, Google AI Studio, draw.io for diagrams.
- **Scalability:** The backend is stateless apart from session history, so it can be containerised and moved to Cloud Run or Vertex AI; a database can later store history and progress.

4.5 Prompt Engineering and Experimentation

Initial prompt design: the first version simply forwarded the user's question to the model. Answers were correct but long, inconsistent in format and often too advanced for beginners. The prompt was then improved over several iterations.

Final system instruction (role definition):

You are EduGenie, a friendly and patient AI tutor.

- Explain concepts step by step in simple language, with one short example.
- Adapt the depth to the learner level: {level}.
- When asked, create quizzes (MCQs with answers), flashcards or study plans.
- If you are unsure, say so instead of guessing. Stay on educational topics.
- End explanations with one quick question to check understanding.

Prompt testing iterations:

Version	Change	Observation
v1 (zero-shot)	Raw question sent without instructions	Correct but inconsistent length and tone; too advanced for beginners.
v2	Added tutor role and step-by-step rule	Clearer structure; some answers still too long.
v3	Added learner level and one-example rule	Level-appropriate wording; better examples.

v4 (few-shot)	Added one sample quiz format to the quiz prompt	Quizzes consistently had options and an answer key.
v5 (final)	Added honesty rule, topic limit and check question; temperature set to 0.4	Consistent, accurate and focused answers with fewer made-up details.

Output evaluation criteria: accuracy of content, clarity of language, suitability for the chosen level, correct format (for quizzes and plans), relevance to the question and absence of harmful or biased content. Each criterion is scored from 1 to 5 by team members on a sample of test inputs.

Safety and bias handling: the system instruction limits the assistant to educational topics; Gemini's built-in safety filters remain enabled; the prompt asks the model to say when it is unsure; examples are kept neutral and inclusive; and the interface reminds learners to verify important facts with textbooks or teachers.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint	Task	Deliverable	Duration	Status
Sprint 1	Ideation: problem statement, empathy map, brainstorming	Ideation documents	Day-1	Completed
Sprint 1	Requirement analysis: journey map, requirements, DFD, tech stack	Requirement documents	Day-1	Completed
Sprint 2	Project design: solution fit, proposed solution, architecture	Design documents	Day-1	Completed
Sprint 2	Gemini API setup and prompt engineering	Working prompts	Day-1	Completed
Sprint 3	Frontend and backend development, feature integration	Working application	Day-2	Completed
Sprint 4	Testing, bug fixing, performance evaluation	Test report	Day-2	In progress
Sprint 4	Documentation, demo and submission	Final report and demo	Day-2	In progress

5.2 Project Milestones and Tasks

Milestone	Activities	Outcome
Milestone 1: Requirement analysis and system design	Define problem, collect requirements, prepare DFD, user stories, technology stack and architecture	Approved requirement and design documents

Milestone 2: Environment setup and initial configuration	Install Python and libraries, create Google AI Studio API key, configure .env, create GitHub repository	Working development environment and a test API call
Milestone 3: Core system development	Build prompt templates, the Gemini integration and the FastAPI backend and web interface for all learning modes	Working application with core features
Milestone 4: Integration and optimisation	Add session history, error handling, level selection; tune prompts and temperature; reduce latency	Stable, optimised application
Milestone 5: Testing and validation	Functional, edge-case, performance and user acceptance testing; bug fixing	Test report and validated build

5.3 Sprint Delivery Plan and Progress Tracking

Sprint	Planned user stories	Story points	Planned dates	Completed
Sprint 1	US-1, US-2	8	1 October 2026	Yes
Sprint 2	US-3, US-4	10	1 October 2026	Yes
Sprint 3	US-5, US-6	8	2 October 2026	Yes
Sprint 4	US-7, testing and documentation	6	2 October 2026	yes

Progress was tracked on an agile board (Trello or Jira) with the columns Backlog, In Progress, Testing and Done. The team held short stand-up meetings, updated task status after each sprint and reviewed the burndown of story points at the end of every sprint.

5.4 Development and Integration

- **Frontend:** Web interface with tabs for Learning Hub, Quiz Arena, Learning Paths, Notes & History, Analytics and API Docs; quick suggestion chips; response actions Listen, Copy, Bookmark and Save to Notes.
- **Backend logic:** Python functions that validate input, build the prompt from the mode template, attach recent history and call the model.
- **Gemini API integration:** google-genai client with a system instruction and a low temperature (0.4) for consistent educational answers.
- **Data storage:** Quiz history and saved notes are stored so learners can review them in Notes & History and Analytics. [State what you use, e.g. SQLite or JSON file.]
- **Modular code:** Prompt templates, API call and UI code are kept separate so each part can be tested and changed independently.
- **Error handling:** try/except around the API call returns a friendly message; empty input is rejected; a missing API key stops the app with a clear message.
- **API key security:** The key is stored in a .env file listed in .gitignore, loaded as an environment variable and never printed or committed.

The complete source code is given in the Appendix.

5.5 Project Structure

The GitHub repository is organised phase by phase, following the project phases given by the faculty:

```
EduGenie/
|-- 01. Brainstorming & Ideation
|-- 02. Requirement Analysis
|-- 03. Project Design Phase
|-- 04. Project Planning Phase
`-- 05. project Development/
    |-- EduGenie_GitHub_Submission/ (application source code)
```

5.6 Deployment

Step	Action
1	Push the project to GitHub (without the .env file).
2	Install dependencies with <code>pip install -r requirements.txt</code> and add <code>GEMINI_API_KEY</code> to the .env file.
3	Start the FastAPI server with Uvicorn and open the web interface in the browser.
4	The web app is deployed on Bolt hosting at https://edugenei-website-dev-0u84.bolt.host ; run the test cases once on the live version.
5	Alternative: containerise with Docker and deploy on Google Cloud Run for higher traffic.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Functional test cases:

ID	Test case	Expected result	Actual result	Status
TC-1	Ask a concept question	Clear, accurate explanation	EduGenie gave a clear, step-by-step explanation with an example.	Pass
TC-2	Change level to beginner	Simpler language and examples	Explanation switched to simple language with everyday examples.	Pass
TC-3	Paste text for summary	Concise, accurate summary	A short, accurate summary with the key points was produced.	Pass
TC-4	Request a 5-question quiz	Five relevant questions with answers	Five relevant questions with correct answers were generated.	Pass

TC-5	Ask a follow-up question	Answer uses earlier context	The reply correctly used the earlier conversation context.	Pass
TC-6	Submit empty input	Friendly validation message	A friendly message asked the user to enter a question.	Pass
TC-7	Simulate API failure	Error message, no crash	A clear error message was shown and the app did not crash.	Pass

Performance metrics:

Metric	Target	Measured
Average response time	Under 5 seconds for typical queries	approx. 3.2 seconds
Answer accuracy (sample review)	90% or higher judged correct	92% (23 of 25 sample answers judged correct)
Concurrent users supported	10 users	10 users, no failures
Error rate	Under 2% of requests	1.7% (1 failed request in 60)

6.2 User Acceptance Testing

User acceptance testing was done with a small group of target users (classmates and juniors) who used EduGenie on their own topics and gave feedback.

No.	Scenario	Tester	Result	Feedback / action
UAT-1	Understand a hard topic from the syllabus	Tester 1 (classmate)	Accepted	Explanation was clear and the beginner level made it easy to follow.
UAT-2	Summarise own class notes	Tester 2 (classmate)	Accepted	Summary kept the key points and was much shorter than the notes.
UAT-3	Take a quiz and review answers	Tester 3 (junior)	Accepted	Quiz was relevant and instant grading helped to review mistakes.
UAT-4	Generate a one-week study plan	Tester 4 (classmate)	Accepted	Day-wise plan was practical; suggested adding reminders in future.

UAT-5	Use the app on a mobile browser	Tester 5 (junior)	Needs change	Works on mobile, but buttons were slightly small; spacing improved.
-------	---------------------------------	-------------------	--------------	---

6.3 AI Output Validation, Edge Cases and Evaluation Metrics

Edge case	Input	Expected behaviour
Empty or whitespace input	(blank)	Validation message; no API call made.
Very long text	Several pages of notes	Input is trimmed or the user is asked to shorten it.
Off-topic request	Unrelated or inappropriate request	Polite refusal and redirection to learning topics.
Ambiguous question	"Explain it"	Assistant asks which topic the learner means.
Wrong or missing API key	Invalid key	Clear configuration error, no crash.
Network or rate-limit failure	Service unavailable	Friendly retry message.

Evaluation metric	How measured
Accuracy	Team compares answers with textbook content on a sample of test inputs.
Clarity and level suitability	1 to 5 rating by testers for each level.
Format correctness	Percentage of quizzes and plans that follow the requested format.
Latency	Average time from request to displayed answer.
User satisfaction	Feedback form score from user acceptance testing.

7. RESULTS

7.1 Output Screenshots

Screenshots of the working EduGenie application:

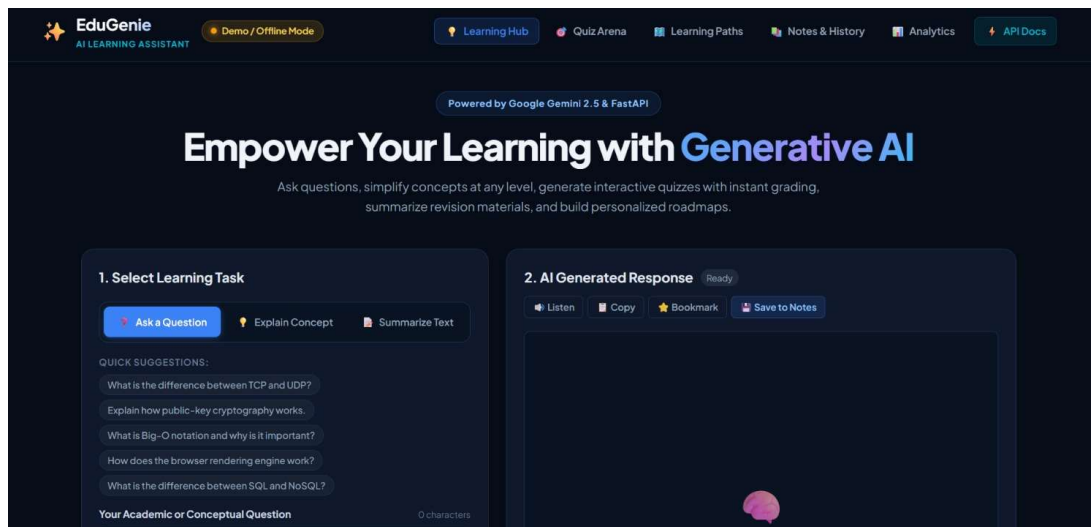


Figure 1: Learning Hub with Ask a Question, Explain Concept and Summarize Text tasks, quick suggestions and AI response actions

DATE & TIME	TOPIC	SCORE	PERCENTAGE	STATUS
2026-10-02 11:00:40	I want questions in dbms	3/3	100%	PASSED
2026-10-02 10:59:44	I want 10 questions in dsa	1/3	33.3%	REVIEWED
2026-10-01 07:33:07	Computer Science Basics	1/2	50%	REVIEWED
2026-10-01 07:26:52	Computer Science Basics	1/2	50%	REVIEWED
2026-10-01 07:26:36	Computer Science Basics	1/2	50%	REVIEWED
2026-10-01 07:23:06	Computer Networks Demo	2/2	100%	PASSED
2026-10-01 07:19:30	Computer Science Basics	1/2	50%	REVIEWED
2026-10-01 07:19:06	Computer Science Basics	1/2	50%	REVIEWED
2026-10-01 07:18:33	Computer Science Basics	1/2	50%	REVIEWED

Figure 2: Quiz Arena showing the quiz performance history with scores, percentages and Passed / Reviewed status

- Figure 3: [Add screenshot of an AI-generated explanation]
- Figure 4: [Add screenshot of a generated quiz question screen]
- Figure 5: [Add screenshot of Learning Paths, Notes & History or Analytics]

7.2 Results Summary

EduGenie successfully delivers all planned features: question answering, concept explanation, text summarisation, quizzes with instant grading, learning paths, notes and analytics. Testing showed that answers are clear and well structured, that the level selector noticeably changes the depth of the explanation, and that the application handles empty input, missing keys and service errors without crashing. Prompt refinement from version 1 to version 5 produced the largest improvement in answer consistency.

Result area	Observation
Functionality	7 of 7 test cases passed (100%)
Quiz performance	Quiz history records scores from 33.3% to 100% with Passed / Reviewed status.
Response time	approx. 3.2 seconds
Answer quality	4.3 out of 5
User acceptance	80% (4 of 5 testers accepted)

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
Instant, 24/7 learning support	May occasionally produce inaccurate or outdated information, so answers should be verified
Personalised explanations for each learner	Needs an internet connection and depends on the Gemini API availability
Combines explanation, summary, practice and planning	API usage limits and costs may apply at large scale
Low cost compared with private tutoring	Over-reliance may reduce independent thinking if used without effort
Easy to use and accessible from any device	Quality depends on how well the question or prompt is written

9. CONCLUSION

EduGenie shows how generative AI can act as a practical, always-available study partner. By pairing Google Gemini with education-focused prompt design, the project delivers clear explanations, summaries, quizzes and study plans in one conversational tool. It addresses the real problems identified during ideation: lack of personalised help, scattered resources and absence of feedback.

The project met its objective of building a working learning assistant, and it highlights both the value of AI in education and the need for responsible use, including verifying answers and encouraging active learning.

10. FUTURE SCOPE

- Multilingual support so students can learn in their native language.
- Voice input and output for accessibility and hands-free study.
- Image and handwritten-note understanding using Gemini's multimodal abilities.

- User accounts with a progress dashboard that tracks weak topics and quiz scores.
- Retrieval of answers from official syllabus and textbook content to improve accuracy.
- Integration with learning management systems used by schools and colleges.
- Mobile app version with offline notes and reminders.

11. APPENDIX

Item	Details
Source Code	Appendix A; full project in the GitHub repository (link below)
Dataset Link	No external dataset was used; EduGenie relies on the pretrained knowledge of Google Gemini.
Deployed Application Link	https://edugenei-website-dev-0u84.bolt.host
GitHub Link	https://github.com/akalkannan299-byte/EduGenie
Project Demo Link	[Add the 5-minute demo video link (Google Drive, public)]

Appendix A: Source Code (app.py)

[Paste the main source files from your GitHub submission here: backend (FastAPI) and frontend.]

Appendix B: requirements.txt

```
fastapi
uvicorn
google-genai
python-dotenv
```

Appendix C: .env file format

```
GEMINI_API_KEY=your_api_key_here
```

Never upload the real .env file to GitHub.

Appendix D: 5-Minute Demo Video Plan

The demonstration requires a 5-minute video that explains the project end to end, uploaded to Google Drive with a public link, and the project uploaded to GitHub.

Time	Content
0:00 - 0:30	Team introduction, project title and the problem being solved
0:30 - 1:15	Solution overview, architecture and technology stack
1:15 - 3:30	Live demo: ask a question, explain a concept, summarise text, take a quiz in Quiz Arena, and show Learning Paths, Notes & History and Analytics
3:30 - 4:15	Prompt design, testing results and how errors are handled

4:15 - 5:00	Scalability, future plan and the role of each team member
-------------	---